

COP2034C Module 3 Programming Assignment

Instructions

In this assignment, we will expand the Palmer Penguins program from Module 2 by organizing related data into **lists**, accessing list elements using **indexes**, using Python **sequence functions and methods**, and improving the program's output with **formatted string literals (f-strings)**.

We will continue working with the three species represented in the Palmer Penguins dataset:

- Chinstrap
- Gentoo
- Adelie

In Module 2, we stored the species names and counts in separate constants and calculated several basic statistics. In this module, we will begin organizing those related values into collections. This is an important step toward working with larger datasets.

We will also replace the string concatenation used in earlier assignments with f-strings and use format specifications to control how floating-point values appear in the output. As we work, we will continue to develop our program incrementally: add a small amount of code, run it, verify that it works correctly, and then continue.

Development Environment

This assignment Complete will be completed in Google Colab, beginning with the Palmer Penguins program we completed for Module 2.

After your program executes correctly:

1. Download your notebook as a Python (.py) file.
2. Upload the .py file to your forked GitHub repository.
3. Take a screenshot showing the successful execution of your program in Google Colab.
4. Save the screenshot as a .png or .jpg file.
5. Upload the screenshot to your GitHub repository.

Do not submit the .ipynb notebook.

1. Start with Your Module 2 Solution

Open the Palmer Penguins notebook you completed for Module 2. Rename the notebook for Module 3. A suggested name is:

PalmerPenguinsM3.ipynb

The Python file you eventually download should have a meaningful name such as:

PalmerPenguinsM3.py

2. Update the ID Header

Update the four-line ID header at the top of your program:

```
# PalmerPenguinsM3.py
# Your Name
# Submission Date
# Uses lists and formatted output to analyze Palmer Penguin statistics.
```

Leave a blank line after the header.

3. Retain the Existing Species and Count Constants

Keep the constants from Module 2 representing the three species:

```
SP_CHINSTRAP = "Chinstrap"
SP_GENTOO = "Gentoo"
SP_ADELIE = "Adelie"
```

Also retain the constants representing the number of observations for each species:

```
NUM_CHINSTRAP = 68
NUM_GENTOO = 123
NUM_ADELIE = 151
```

We will use these constants to build lists containing related values.

The **TOTAL_SPECIES** constant can be removed. In this assignment, Python will determine the number of species from the list size instead of storing the value separately; this provides more flexibility for our application.

4. Create a List of Species Names

A list allows multiple related values to be stored in one variable. Create a list named `species_names` containing the three species constants:

```
species_names = [
    SP_CHINSTRAP,
    SP_GENTOO,
    SP_ADELIE
]
```

The elements have the following indexes:

```
Index 0: Chinstrap
Index 1: Gentoo
Index 2: Adelie
```

Remember that Python list indexes begin with 0.

5. Create a List of Species Counts

Create a second list named `species_counts` containing the three count constants:

```
species_counts = [  
    NUM_CHINSTRAP,  
    NUM_GENTOO,  
    NUM_ADELIE  
]
```

These two lists are **parallel lists**, meaning that values at the same index are related. For example, `species_names[0]` contains "Chinstrap" and `species_counts[0]` contains 68:

```
species_names[0] Chinstrap  
species_counts[0] 68
```

Also:

```
species_names[2] Adelie  
species_counts[2] 151
```

Using the same index allows the program to access related values stored in separate lists.

6. Determine the Number of Species Using `len()`

Instead of assigning the value 3 directly, we can use Python's `len()` function to determine how many elements are stored in `species_names`. Create a variable named `total_species`:

```
total_species = len(species_names)
```

The `len()` function returns the number of elements stored in a sequence such as a string or list. We use a variable here instead of a constant since the result can vary now.

7. Calculate the Total Number of Penguins Using `sum()`

Modify the Module 2 calculation for `total_penguins`. Instead of adding the three individual constants, use Python's `sum()` function:

```
total_penguins = sum(species_counts)
```

The `sum()` function adds the numeric elements stored in a list. The result should still be 342.

8. Calculate the Average Number of Penguins per Species

Continue calculating the average number of observations per species. Use the values calculated from the lists:

```
average_penguins = total_penguins / total_species
```

Continue using `math.ceil()` from Module 2 to calculate the rounded-up average:

```
average_penguins_rounded_up = math.ceil(average_penguins)
```

Keep the existing `import math` statement at the beginning of your program under the ID header.

9. Calculate the Species Percentages Using List Indexes

Continue calculating the percentage represented by each species, but modify the expressions so that they retrieve the counts from `species_counts`. For example:

```
chinstrap_percent = species_counts[0] / total_penguins * 100
```

Create similar calculations for Gentoo and Adelie using indexes 1 and 2. For this assignment, use the list indexes explicitly instead of a loop. The purpose is to practice accessing individual list elements by position.

10. Find the Species with the Most and Fewest Observations

Python provides `max()` and `min()` functions for finding the largest and smallest values in a numeric list. Create the following variables:

```
largest_count = max(species_counts)  
smallest_count = min(species_counts)
```

Next, use the `index()` list method to determine where those values occur:

```
largest_index = species_counts.index(largest_count)  
smallest_index = species_counts.index(smallest_count)
```

Use those indexes to retrieve the corresponding species names:

```
largest_species = species_names[largest_index]  
smallest_species = species_names[smallest_index]
```

For this dataset, the results should identify:

- Adelie as having the most observations.
- Chinstrap as having the fewest.
- Gentoo in between Adelie and Chinstrap.

11. Replace Concatenation with f-Strings

In Module 2, our output displayed numeric values using string concatenation and string conversion. For example:

```
print("There are a total of " +  
      str(total_penguins) +  
      " penguins in the dataset.")
```

This module introduced formatted string literals, known as **f-strings**. For this assignment, replace output statements that contain variables with f-strings. For example:

```
print(f"There are a total of {total_penguins} penguins in the dataset.")
```

An f-string begins with the letter f before the opening quotation mark. Expressions inside braces { } are evaluated and inserted into the string, so we no longer need to call str() simply to include numbers in the output. Static output that contains no variables may continue to use normal strings.

12. Format Floating-Point Output

Module 2 intentionally displayed the full floating-point percentage values. In Module 3, we use f-string **format specifications** to control the number of decimal places displayed. Display the average with one decimal place:

```
print(f"The average number of penguins per species is "  
      f"{average_penguins:.1f}.")
```

Display each species percentage with two decimal places. For example:

```
print(f"{species_names[0]}: {species_counts[0]} "  
      f"({chinstrap_percent:.2f}%)")
```

The format specification **“.2f”** is used to display the floating-point number using two digits after the decimal point.

13. Display the Species with the Most and Fewest Observations

After displaying the individual species statistics, add two additional lines identifying the species with the most and fewest observations. Use the variables calculated previously. For example, your output should specify the following:

```
The species with the most observations is Adelie with 151 penguins.  
The species with the fewest observations is Chinstrap with 68 penguins.
```

Use f-strings to produce these lines.

Expected Output

Your program should produce output similar to the following:

```
Introducing the Palmer Penguins:
```

```
Chinstrap!  
Gentoo!  
and last but not least...
```

Adelie!

There are a total of 3 penguin species in this dataset.

There are a total of 342 penguins in the dataset.

The average number of penguins per species is 114.0.

The average rounded up is 114 penguins.

Chinstrap: 68 (19.88%)

Gentoo: 123 (35.96%)

Adelie: 151 (44.15%)

The species with the most observations is Adelie with 151 penguins.

The species with the fewest observations is Chinstrap with 68 penguins.

Minor whitespace differences are acceptable as long as the output is readable and contains all required information.

Data Science Connection

Lists provide a way to organize collections of related data rather than storing every value in a separate variable. In this assignment, the **species_names** list stores the categories in the dataset, while **species_counts** stores a numeric value associated with each category.

Using functions such as:

```
len()
sum()
min()
max()
```

allows programs to calculate descriptive information directly from collections of data.

These ideas will become increasingly important when we begin reading and analyzing the Palmer Penguins dataset directly with Python.

Readability

In addition to whether your program runs correctly, your assignments are graded on code readability. We write code not just for our own use, but for others to read and understand. Keep your program readable by:

- using meaningful variable names,
- using consistent whitespace around operators,
- separating logical sections with blank lines,
- keeping lines reasonably short,

- adding comments where appropriate,
- formatting lists so their contents are easy to read,
- following the Python naming conventions demonstrated in this assignment.

Use comments to identify logical sections of your program, such as:

```
# constants representing the species and counts
# lists containing related penguin data
# calculate dataset statistics
# determine largest and smallest species groups
# output formatted statistics
```

Screenshot

Run the completed program in Google Colab. Take a screenshot showing the program's output and successful execution. The screenshot does not need to include all of your source code because the .py file will be reviewed separately. Save the screenshot as a .png or .jpg file.

The suggested filename is “**PalmerPenguinsM3.png**”.

Submission

Upload the following two files to your forked GitHub repository:

- PalmerPenguinsM3.py
- Screenshot of the program executing successfully (.png or .jpg)

Submit the URL of your forked GitHub repository to the Canvas assignment. **Do not submit the Google Colab .ipynb notebook.**